

Day 6 - Giới thiệu React & JSX

Mục tiêu: Chuyển từ backend Node.js sang frontend React — hiểu tại sao React tồn tại, thiết lập môi trường, viết component đầu tiên với JSX, truyền dữ liệu qua props.

Sau Day 6, thực tập sinh có thể:

- Giải thích React là gì và tại sao cần Virtual DOM
- Khởi tạo project React bằng Vite
- Viết function component trả về JSX hợp lệ
- Phân biệt JSX với HTML — những điểm khác biệt quan trọng
- Truyền dữ liệu từ component cha xuống component con qua props
- Render danh sách bằng `.map()` với `key` prop đúng chuẩn
- Tổ chức component thành file riêng và import/export đúng cách
- Phân biệt `useState` và biến `let` khi cập nhật giá trị trong React
- Sử dụng đúng `useState`, `useRef`, `useMemo`, `useCallback`
- Hiểu khi nào dùng hook nào để tối ưu performance

Câu hỏi tìm hiểu trước

Trước khi học, tìm hiểu và trả lời các câu hỏi sau. Không cần đúng hoàn toàn — mục tiêu là có hình dung ban đầu trước khi đào sâu.

Vấn đề của web truyền thống

- Trước React (jQuery/vanilla JS), việc cập nhật UI khi data thay đổi được làm thế nào?
- Vấn đề gì xảy ra khi cập nhật DOM thủ công ở nhiều nơi trong codebase lớn?
- Single Page Application (SPA) là gì? Khác website truyền thống (Multi Page Application) ở điểm nào?

React cơ bản

- React là gì? Do ai tạo ra, dùng để làm gì?
- Virtual DOM là gì? Tại sao nó nhanh hơn DOM thật?
- Component trong React là gì? Tại sao lại chia UI thành components?
- React library hay framework? Sự khác biệt là gì?

JSX

- JSX là gì? Nó là HTML không? Browser có hiểu JSX trực tiếp không?
- Babel và transpiling là gì? Vai trò của nó trong React project?
- Tại sao phải `import React from 'react'` trong các phiên bản cũ?

Vite

- Vite là gì? Khác Create React App (CRA) ở điểm nào?
- `npm run dev` và `npm run build` khác nhau thế nào?
- Hot Module Replacement (HMR) là gì?

Phần 0 - Chuẩn bị

Kiểm tra môi trường:

```
node --version    # >= 18.x
npm --version     # >= 9.x
```

Khởi tạo project React với Vite:

```
npm create vite@latest day6-react -- --template react
cd day6-react
npm install
npm run dev
```

Mở `http://localhost:5173` — thấy trang Vite + React mặc định là môi trường OK.

Cấu trúc project Vite + React:

```
day6-react/
├── public/           # static files (favicon, ảnh tĩnh)
├── src/
│   ├── assets/      # ảnh, font dùng trong code
│   ├── App.jsx      # component gốc
│   ├── App.css
│   ├── main.jsx     # entry point – ReactDOM.createRoot()
│   └── index.css
├── index.html       # HTML shell – React mount vào <div id="root">
├── vite.config.js
└── package.json
```

Xem `src/main.jsx` :

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";
import "./index.css";

// ReactDOM tìm div#root trong index.html và "mount" App vào đó
ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);
```

`React.StrictMode` không thay đổi output — nó kích hoạt cảnh báo trong development để phát hiện vấn đề sớm.

Phần 1 - JSX là gì?

Vấn đề JSX giải quyết

Trước JSX, code React trông như thế này:

```
// Không có JSX – khó đọc, khó maintain
const element = React.createElement(
  "div",
  { className: "card" },
  React.createElement("h2", null, "Alice"),
  React.createElement("p", null, "alice@example.com")
);
```

JSX cho phép viết cú pháp giống HTML bên trong JavaScript:

```
// Với JSX – rõ ràng hơn nhiều
const element = (
  <div className="card">
    <h2>Alice</h2>
    <p>alice@example.com</p>
  </div>
);
```

Babel compile JSX thành `React.createElement()` — browser không hiểu JSX trực tiếp.

JSX khác HTML — những điểm quan trọng

```
// 1. class -> className (vì "class" là từ khóa JS)
<div className="container">

// 2. for -> htmlFor (vì "for" là từ khóa JS)
<label htmlFor="email">Email</label>
<input id="email" />

// 3. Thuộc tính camelCase
<button onClick={handleClick}> // không phải onclick
<input onChange={handleChange} // không phải onchange
<div style={{ fontSize: 16, color: "red" }}> // style là object, không phải string

// 4. Tag đơn phải tự đóng
 // không phải 
<br />
<input type="text" />

// 5. Chỉ một root element (hoặc dùng Fragment)
// SAI:
return (
  <h1>Title</h1>
  <p>Content</p>
);

// ĐÚNG – wrap trong một element:
return (
  <div>
    <h1>Title</h1>
```

```

    <p>Content</p>
  </div>
);

// ĐÚNG – hoặc dùng Fragment (không tạo thêm DOM node):
return (
  <>
    <h1>Title</h1>
    <p>Content</p>
  </>
);

```

Biểu thức trong JSX

Dùng `{}` để nhúng bất kỳ biểu thức JavaScript nào vào JSX:

```

const name = "Alice";
const score = 85;
const isAdmin = true;

function UserCard() {
  return (
    <div>
      {/* Comment trong JSX */}
      <h2>{name}</h2>                {/* biến */}
      <p>Score: {score * 2}</p>        {/* expression */}
      <p>Status: {score >= 70 ? "Pass" : "Fail"}</p>  {/* ternary */}
      {isAdmin && <span>Admin</span>}           {/* conditional render */}
      <p>{new Date().toLocaleDateString()}</p>      {/* function call */}
    </div>
  );
}

```

Không thể dùng trong `{}` : `if` , `for` , `switch` — chỉ được expression (có giá trị trả về), không phải statement.

Phần 2 - Function Component

Component là gì?

Analogous với backend: component giống như một **controller/view** — nhận input (props), trả về output (UI). Khác ở chỗ output là JSX thay vì JSON.

```

Backend: request → controller → JSON response
Frontend: props  → component  → JSX (UI)

```

Quy tắc function component

```
// src/components/UserCard.jsx
```

```
// 1. Tên phải viết hoa chữ đầu (PascalCase)
//   React phân biệt <div> (HTML tag) và <UserCard> (component) bằng chữ hoa
function UserCard() {
  // 2. Phải return JSX (hoặc null nếu không render gì)
  return (
    <div className="user-card">
      <h2>Alice</h2>
      <p>alice@example.com</p>
    </div>
  );
}

// 3. Export để dùng ở file khác
export default UserCard;
```

Named export vs Default export

```
// Default export – mỗi file chỉ có 1
export default function UserCard() { ... }
// Import: import UserCard from "./UserCard"; (tên tùy ý)
// Import: import MyCard from "./UserCard"; (cũng được)

// Named export – một file có thể có nhiều
export function UserCard() { ... }
export function UserAvatar() { ... }
// Import: import { UserCard, UserAvatar } from "./UserCard"; (phải đúng tên)
```

Quy ước trong project: Mỗi component một file, dùng default export.

Tổ chức component

```
src/
├── components/
│   ├── UserCard.jsx
│   ├── UserList.jsx
│   └── Header.jsx
├── App.jsx
└── main.jsx
```

```
// src/App.jsx
import UserCard from "./components/UserCard";
import Header from "./components/Header";

function App() {
  return (
    <div>
      <Header />
      <UserCard />
    </div>
  );
}
```

```
}  
  
export default App;
```

Phần 3 - Props

Props là gì?

Props (properties) là cách truyền dữ liệu từ component **cha** xuống component **con**. Props là **read-only** — component con không được tự sửa props nhận được.

Analogy backend: props giống như query params hoặc request body
mà route handler nhận vào — chỉ đọc, không sửa

Truyền và nhận props

```
// src/components/UserCard.jsx  
  
// Props được truyền vào như tham số đầu tiên — thường destructure ngay  
function UserCard({ name, email, role }) {  
  return (  
    <div className="user-card">  
      <h2>{name}</h2>  
      <p>{email}</p>  
      <span className={`badge badge-${role}`}>{role}</span>  
    </div>  
  );  
}  
  
export default UserCard;
```

```
// src/App.jsx  
import UserCard from "../components/UserCard";  
  
function App() {  
  return (  
    <div>  
      {/* Truyền props giống như HTML attributes */}  
      <UserCard name="Alice" email="alice@example.com" role="admin" />  
      <UserCard name="Bob" email="bob@example.com" role="user" />  
    </div>  
  );  
}
```

Props với nhiều kiểu dữ liệu

```
function ProductCard({ name, price, inStock, tags, onAddToCart }) {  
  return (  

```

```

    <div>
      <h3>{name}</h3>
      <p>{price.toLocaleString()} VND</p>          {/* number */}
      <p>{inStock ? "Còn hàng" : "Hết hàng"}</p>    {/* boolean */}
      <ul>
        {tags.map((tag) => <li key={tag}>{tag}</li>)} {/* array */}
      </ul>
      {/* function prop – truyền callback */}
      <button onClick={onAddToCart} disabled={!inStock}>
        Thêm vào giỏ
      </button>
    </div>
  );
}

// Dùng:
<ProductCard
  name="Laptop XYZ"
  price={25000000}
  inStock={true}
  tags={["laptop", "gaming", "sale"]}
  onAddToCart={() => console.log("Added!")}
/>

```

Default props

```

// Cách 1: Default value trong destructuring (khuyến khích)
function UserCard({ name = "Anonymous", role = "user", avatar = "/default-
avatar.png" }) {
  return <div>...</div>;
}

// Cách 2: defaultProps (cũ hơn, vẫn gặp trong codebase cũ)
UserCard.defaultProps = {
  name: "Anonymous",
  role: "user",
};

```

Props children

```

// children là prop đặc biệt – nội dung nằm giữa opening/closing tag
function Card({ title, children }) {
  return (
    <div className="card">
      <h2 className="card-title">{title}</h2>
      <div className="card-body">
        {children} {/* render bất cứ thứ gì được wrap bên trong */}
      </div>
    </div>
  );
}

```

```
    );  
  }  
  
  // Dùng:  
<Card title="Thông tin người dùng">  
  <p>Tên: Alice</p>  
  <p>Email: alice@example.com</p>  
  <button>Chỉnh sửa</button>  
</Card>
```

Phần 4 - Render danh sách

.map() để render list

```
const users = [  
  { id: 1, name: "Alice", email: "alice@example.com" },  
  { id: 2, name: "Bob", email: "bob@example.com" },  
  { id: 3, name: "Charlie", email: "charlie@example.com" },  
];  
  
function UserList() {  
  return (  
    <ul>  
      {users.map((user) => (  
        // key prop BẮT BUỘC – giúp React track thay đổi trong list  
        // Dùng ID từ data (không dùng index nếu list có thể thay đổi thứ tự)  
        <li key={user.id}>  
          {user.name} – {user.email}  
        </li>  
      ))}  
    </ul>  
  );  
}
```

Tại sao key quan trọng?

Không có key:

React không biết phần tử nào thay đổi -> re-render toàn bộ list
-> chậm, mất state của từng item

Có key:

React dùng key để map "element cũ" với "element mới"
-> chỉ update phần tử thực sự thay đổi -> nhanh hơn

```
// SAI – dùng index làm key khi list có thể sort/filter/xóa  
{users.map((user, index) => (  
  <li key={index}>{user.name}</li> // đừng làm thế này  
))}
```

```
// ĐÚNG – dùng unique ID từ data
{users.map((user) => (
  <li key={user.id}>{user.name}</li>
))}

// Nếu data không có ID (rare): dùng thư viện uuid hoặc nanoid
import { nanoid } from "nanoid";
const itemWithId = { ...item, id: nanoid() };
```

Render component trong list

```
function UserList({ users }) {
  if (users.length === 0) {
    return <p>Không có người dùng nào.</p>;
  }

  return (
    <div className="user-list">
      {users.map((user) => (
        // Truyền toàn bộ user object qua spread, thêm key ở ngoài
        <UserCard key={user.id} {...user} />
        // Tương đương: <UserCard key={user.id} name={user.name} email={user.email}
      ))}
    </div>
  );
}
```

Phần 5 - React Hooks cơ bản

Tại sao cần Hooks?

Hooks cho phép function component có **state** và **side effect** — trước React 16.8, chỉ class component mới làm được điều này.

Quy tắc quan trọng:

- Chỉ gọi hooks ở **top level** của component (không trong if/for/nested function)
- Chỉ gọi hooks trong **React function component** hoặc **custom hook**

5.1 — useState: Quản lý state

setState vs **let** — Tại sao **let** không hoạt động?

```
// SAI – dùng let: UI KHÔNG cập nhật khi click
function Counter() {
  let count = 0;

  function handleClick() {
    count += 1;
  }
}
```

```

    console.log(count); // giá trị tăng trong console
    // nhưng UI vẫn hiển thị 0 – React không biết cần re-render!
  }

  return <button onClick={handleClick}>Count: {count}</button>;
}

```

```

// ĐÚNG – dùng useState: UI cập nhật khi click
import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1); // React biết state thay đổi → re-render component
  }

  return <button onClick={handleClick}>Count: {count}</button>;
}

```

Tại sao `let` không work?

```

let count = 0;
→ Mỗi lần React re-render component, function chạy lại từ đầu
→ count reset về 0
→ React KHÔNG theo dõi biến let → không trigger re-render

```

```

useState(0):
→ React lưu giá trị state BÊN NGOÀI function
→ Khi gọi setCount(), React:
  1. Cập nhật giá trị trong bộ nhớ nội bộ
  2. Gọi lại function component (re-render)
  3. useState() trả về giá trị mới
→ UI luôn đồng bộ với state

```

State là immutable — không mutate trực tiếp

```

// SAI – mutate object/array trực tiếp
const [user, setUser] = useState({ name: "Alice", age: 25 });
user.name = "Bob"; // React không phát hiện thay đổi
setUser(user); // cùng reference → không re-render

// ĐÚNG – tạo object mới
setUser({ ...user, name: "Bob" });

// SAI – mutate array
const [items, setItems] = useState(["a", "b"]);
items.push("c"); // mutate gốc
setItems(items); // cùng reference → không re-render

```

```
// ĐÚNG – tạo array mới
setItems([...items, "c"]); // thêm
setItems(items.filter(i => i !== "a")); // xóa
setItems(items.map(i => i === "b" ? "B" : i)); // sửa
```

Functional update — khi state mới phụ thuộc state cũ

```
// Có thể sai nếu gọi setCount nhiều lần liên tiếp
setCount(count + 1);
setCount(count + 1); // count vẫn là giá trị cũ → chỉ +1, không +2

// ĐÚNG – dùng callback form
setCount(prev => prev + 1);
setCount(prev => prev + 1); // +2 đúng vì prev luôn là giá trị mới nhất
```

5.2 — useRef: Giữ giá trị không gây re-render

`useRef` tạo một "hộp" chứa giá trị `.current` — thay đổi nó **không trigger re-render**.

Hai use case chính:

```
import { useRef, useState } from "react";

// Use case 1: Truy cập DOM element
function TextInput() {
  const inputRef = useRef(null);

  function handleFocus() {
    inputRef.current.focus(); // truy cập DOM node trực tiếp
  }

  return (
    <div>
      <input ref={inputRef} type="text" />
      <button onClick={handleFocus}>Focus vào input</button>
    </div>
  );
}

// Use case 2: Lưu giá trị giữa các lần render mà không gây re-render
function Stopwatch() {
  const [time, setTime] = useState(0);
  const intervalRef = useRef(null); // lưu interval ID

  function start() {
    intervalRef.current = setInterval(() => {
      setTime(prev => prev + 1);
    }, 1000);
  }
}
```

```
function stop() {
  clearInterval(intervalRef.current); // dùng ref để clear
}

return (
  <div>
    <p>{time}s</p>
    <button onClick={start}>Start</button>
    <button onClick={stop}>Stop</button>
  </div>
);
}
```

So sánh useState vs useRef vs let

Đặc điểm	let	useRef	useState
Giữ giá trị qua các lần render	x reset mỗi lần render	✓ giữ nguyên	✓ giữ nguyên
Thay đổi gây re-render?	x	x	✓ re-render
Dùng khi	Biến tạm trong render	DOM ref, timer ID, giá trị không cần UI	Dữ liệu hiển thị trên UI

5.3 — useMemo: Cache kết quả tính toán nặng

`useMemo` lưu kết quả một phép tính — chỉ tính lại khi dependencies thay đổi.

```
import { useState, useMemo } from "react";

function ProductList({ products }) {
  const [search, setSearch] = useState("");
  const [sortBy, setSortBy] = useState("name");

  // Không có useMemo: filter + sort chạy MỖI LẦN component re-render
  // (kể cả khi products và search không đổi)

  // Có useMemo: chỉ tính lại khi products, search, hoặc sortBy thay đổi
  const filteredProducts = useMemo(() => {
    console.log("Đang filter và sort..."); // log này chỉ xuất hiện khi deps thay đổi
    return products
      .filter(p => p.name.toLowerCase().includes(search.toLowerCase()))
      .sort((a, b) => {
        if (sortBy === "name") return a.name.localeCompare(b.name);
        if (sortBy === "price") return a.price - b.price;
        return 0;
      });
  }, [products, search, sortBy]); // dependency array
```

```

return (
  <div>
    <input value={search} onChange={e => setSearch(e.target.value)}
placeholder="Tìm kiếm..." />
    <select value={sortBy} onChange={e => setSortBy(e.target.value)}>
      <option value="name">Theo tên</option>
      <option value="price">Theo giá</option>
    </select>
    {filteredProducts.map(p => <ProductCard key={p.id} {...p} />)}
  </div>
);
}

```

Khi nào dùng useMemo:

- Tính toán nặng (filter/sort list lớn, format phức tạp)
- Tạo object/array truyền xuống child component (tránh re-render không cần thiết)

Khi nào KHÔNG cần:

- Phép tính đơn giản (`a + b` , string concatenation)
- Component ít re-render

5.4 — useCallback: Cache function reference

`useCallback` trả về cùng function reference giữa các lần render — tránh child component re-render vì nhận prop function mới mỗi lần.

```

import { useState, useCallback } from "react";

function App() {
  const [count, setCount] = useState(0);
  const [text, setText] = useState("");

  // Không có useCallback: handleAdd được tạo MỚI mỗi lần App re-render
  // → ExpensiveList nhận prop mới → re-render (dù data không đổi)

  // Có useCallback: handleAdd giữ nguyên reference nếu deps không đổi
  const handleAdd = useCallback((item) => {
    setCount(prev => prev + 1);
    console.log("Added:", item);
  }, []); // [] = function không thay đổi

  const handleSearch = useCallback((query) => {
    console.log("Search:", query);
  }, []); // không phụ thuộc gì → stable reference

  return (
    <div>
      <input value={text} onChange={e => setText(e.target.value)} />
      <p>Count: {count}</p>
      {/* Khi user gõ vào input → App re-render

```

```

        → nhưng handleAdd reference không đổi → ExpensiveList KHÔNG re-render */}
        <ExpensiveList onAdd={handleAdd} onSearch={handleSearch} />
    </div>
  );
}

```

Mối quan hệ useMemo và useCallback:

```

// useCallback(fn, deps) tương đương useMemo(() => fn, deps)

// useMemo: cache GIÁ TRỊ (kết quả function trả về)
const sorted = useMemo(() => items.sort(), [items]);

// useCallback: cache FUNCTION (chính function đó)
const handleClick = useCallback(() => { doSomething() }, []);

```

5.5 — useEffect: Side effects (preview)

useEffect dùng cho side effects: fetch API, subscribe, update document title,...

```

import { useState, useEffect } from "react";

function UserProfile({ userId }) {
  const [user, setUser] = useState(null);

  useEffect(() => {
    // Side effect: gọi API
    fetch(`/api/users/${userId}`)
      .then(res => res.json())
      .then(data => setUser(data));

    // Cleanup function (optional): chạy khi component unmount hoặc trước khi effect
    // chạy lại
    return () => {
      console.log("Cleanup for userId:", userId);
    };
  }, [userId]); // Chỉ chạy lại khi userId thay đổi

  if (!user) return <p>Loading...</p>;
  return <h2>{user.name}</h2>;
}

```

Dependency array quyết định khi nào effect chạy:

```

useEffect(() => {...})           // Không có deps → chạy SAU MỖI lần render
useEffect(() => {...}, [])       // [] → chạy MỘT LẦN sau render đầu tiên (mount)
useEffect(() => {...}, [a, b])   // Chạy khi a hoặc b thay đổi

```

5.6 — Tổng hợp: Khi nào dùng hook nào?

Cần gì?	Dùng hook nào
Lưu dữ liệu hiển thị trên UI	useState
Lưu giá trị KHÔNG cần hiển thị	useRef
Truy cập DOM element	useRef
Tính toán nặng, cache kết quả	useMemo
Truyền function xuống child, tránh re-render thừa	useCallback
Gọi API, subscribe, timer	useEffect

Phần 6 - Kết nối với API backend

Thực tập sinh đã biết API từ Day 1-5. Đây là cách component React gọi API đó.

Lưu ý: Phần này chỉ xem trước để thấy React kết nối với backend thế nào — Day 7-8 sẽ học chi tiết `useState` và `useEffect`. Hôm nay dùng data tĩnh (hardcode) trong bài tập.

```
// Preview – sẽ học kỹ ở Day 7-8
import { useState, useEffect } from "react";

function UserList() {
  const [users, setUsers] = useState([]); // state – học Day 7
  const [loading, setLoading] = useState(true);

  useEffect(() => { // lifecycle – học Day 8
    fetch("http://localhost:3000/api/users")
      .then((res) => res.json())
      .then((data) => {
        setUsers(data);
        setLoading(false);
      });
  }, []);

  if (loading) return <p>Đang tải...</p>;

  return (
    <div>
      {users.map((user) => (
        <UserCard key={user._id} name={user.name} email={user.email} />
      ))}
    </div>
  );
}
```

Bài tập trong giờ

Bài tập 1 — JSX Basics

Xem `exercises/01-jsx-basics/App.jsx`

Viết component `ProfileCard` nhận props: `name`, `title`, `bio`, `avatarUrl`. Render đúng JSX, không có lỗi cú pháp.

Bài tập 2 — Props & Component Tree

Xem `exercises/02-props/App.jsx`

Tạo component `ProductCard` nhận props: `name` (string), `price` (number), `category` (string), `inStock` (boolean). Hiển thị giá theo định dạng VND. Nếu hết hàng, disable button "Mua ngay".

Bài tập 3 — Render List

Xem `exercises/03-list/App.jsx`

Cho sẵn array `products` gồm 5 items. Render danh sách dùng `ProductCard` component từ Bài 2. Đảm bảo `key` prop đúng. Thêm empty state khi array rỗng.

Bài tập 4 — useState vs let

Xem `exercises/04-state-vs-let/App.jsx`

Tạo 2 component `CounterWithLet` và `CounterWithState`:

- `CounterWithLet`: dùng `let count = 0`, button click `count++` — quan sát UI không đổi
- `CounterWithState`: dùng `useState(0)`, button click `setCount(prev => prev + 1)` — UI cập nhật
- Thêm `console.log` ở cả 2 để thấy giá trị thực tế
- Viết 3 dòng giải thích tại sao `let` không work trong React

Bài tập 5 — Hooks Practice

Xem `exercises/05-hooks/App.jsx`

Tạo component `SearchableList`:

- Dùng `useState` cho search input và danh sách items
- Dùng `useMemo` để filter list theo search (chỉ tính lại khi search hoặc items đổi)
- Dùng `useCallback` cho hàm `handleRemove` truyền xuống child `Listitem`
- Dùng `useRef` để auto-focus vào search input khi component mount

Homework — Card List UI

Xem `homework/card-list/`

Xây dựng một trang danh sách sản phẩm mini với:

Yêu cầu:

- Component `App` render `ProductList`
- Component `ProductList` nhận array `products` qua props, render danh sách `ProductCard`
- Component `ProductCard` nhận: `id`, `name`, `price`, `category`, `rating` (1-5), `inStock`
- Hiển thị rating dạng sao (★★★★☆) — tính từ số
- Filter: chỉ hiển thị sản phẩm còn hàng (`inStock === true`) — dùng `.filter()` trước `.map()`
- Empty state: nếu không có sản phẩm nào, hiển thị thông báo

Data mẫu (hardcode trong App.jsx):

```
const products = [
  { id: 1, name: "Laptop Dell XPS 13", price: 28000000, category: "laptop", rating: 4, inStock: true },
  { id: 2, name: "Chuột Logitech MX Master", price: 2500000, category: "peripheral", rating: 5, inStock: true },
  { id: 3, name: "Tai nghe Sony WH-1000XM5", price: 8000000, category: "audio", rating: 4, inStock: false },
  { id: 4, name: "Bàn phím Keychron K2", price: 2200000, category: "peripheral", rating: 3, inStock: true },
  { id: 5, name: "Màn hình LG 27UL850", price: 12000000, category: "monitor", rating: 4, inStock: false },
];
```

Cấu trúc file:

```
homework/card-list/
├── src/
│   ├── components/
│   │   ├── ProductCard.jsx
│   │   └── ProductList.jsx
│   ├── App.jsx
│   ├── App.css
│   └── main.jsx
└── package.json
```

Tiêu chí đánh giá (Pass >= 70/100)

Hạng mục	Điểm	Tiêu chí
Exercise 01 — JSX	10	JSX hợp lệ, không lỗi cú pháp, dùng className thay class
Exercise 02 — Props	15	Nhận props đúng type, format giá VND, disabled button khi hết hàng
Exercise 03 — List	15	key prop từ ID (không dùng index), empty state hoạt động
Exercise 04 — setState vs let	10	Giải thích đúng tại sao let không trigger re-render, demo 2 counter
Exercise 05 — Hooks	15	Dùng đúng useState, useMemo, useCallback, useRef trong SearchableList
Homework — filter	15	.filter() hoạt động đúng, chỉ hiện inStock items
Homework — rating	10	Render đúng số sao từ số rating
Cấu trúc file	10	Mỗi component một file, import/export đúng

Fail ngay nếu:

- Dùng class thay vì className (React warning)
- Không có key prop khi render list (React warning)

- Dùng index làm key trong list có thể thay đổi thứ tự
 - Component không export — file khác import không được
 - Dùng `let / var` thay `useState` để lưu giá trị cần hiển thị trên UI
 - Gọi hook bên trong `if / for / nested function` (vi phạm Rules of Hooks)
 - Mutate state trực tiếp (`state.push()` , `state.name = "x"`) thay vì tạo object/array mới
-

Tài liệu tham khảo

Bắt buộc đọc:

- [React - Quick Start](#)
- [React - Describing the UI](#)
- [React - Your First Component](#)
- [React - Passing Props to a Component](#)
- [React - Rendering Lists](#)

Hooks — bắt buộc đọc:

- [React - useState](#)
- [React - useRef](#)
- [React - useMemo](#)
- [React - useCallback](#)
- [React - useEffect](#)
- [React - Rules of Hooks](#)

Đọc thêm:

- [Vite - Getting Started](#)
- [React - JSX In Depth](#)
- [MDN - Array.prototype.map\(\)](#)

Tool:

- [React DevTools](#) — extension Chrome/Firefox để inspect component tree và props
-

Stuck quá 15 phút với một bài -> hỏi mentor ngay.